Lezione 6 – `this`, `static`, `final`, immutabilità

Classi in Java: aspetti avanzati

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

20 ottobre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 5** (*15 min*)
   - uno di voi alla lavagna con `Rettangolo`, `Orario` o `Libretto`: lo commentiamo insieme
2. **Il riferimento `this`; copia e confronto di oggetti** (*20 min*)
   - `this` implicito ed esplicito
   - `=` e `==` sui riferimenti; i metodi `equals` e `notEqual`
3. **Membri `static`, variabili `final`, classi immutabili** (*25 min*)
   - variabili di classe e metodi `static`; variabili di istanza `final`
   - classi immutabili e *privacy leak*
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: `Point2D`, un punto del piano che non cambia mai
   - *Insieme, passo passo*: `Circle` – dal programma di prova alla classe
5. **Wrap-up e Homework 6** (*10 min*)

Il riferimento `this`: su quale oggetto lavora un metodo?

In `Counter`:

```
    public int getValue() {
        return value;
    }
```

Nel `main` di `ProvaCounter`:

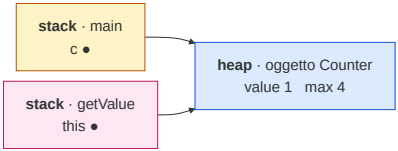
```
Counter c = new Counter(4);
Counter d = new Counter(c);
c.increment();
d.increment();
d.increment();
System.out.println(c.getValue());    // 1
System.out.println(d.getValue());    // 2
```

Nel corpo di `getValue` nulla dice di quale oggetto sia `value`. Eppure `c.getValue()` restituisce il valore di `c`, e `d.getValue()` quello di `d`.

Come fa il metodo a sapere su quale oggetto è stato chiamato?

Il riferimento `this`

In ogni metodo non `static`, e in ogni costruttore, il compilatore aggiunge un riferimento nascosto di nome `this`: si riferisce all'*oggetto corrente*: l'oggetto su cui il metodo è stato chiamato o, in un costruttore, l'oggetto appena creato. `this` si può usare, non modificare.



La memoria durante `c.getValue()`: alla chiamata, `this` riceve una copia del riferimento `c`, come un parametro in più (lezione 3). Durante `d.getValue()`, `this` porta all'oggetto di `d`.

Il riferimento `this`: uso implicito e uso esplicito

Uso implicito. In un metodo non `static`, il nome di una variabile di istanza o di un metodo della classe, scritto da solo, è riferito a `this`.

Scritto nel metodo	Equivale a
`return value;`	`return this.value;`
`value++;`	`this.value++;`
`if (!isValido(voto))`	`if (!this.isValido(voto))`

- è il caso normale: per questo finora non abbiamo mai scritto `this`
- nella lezione 5 `registra` chiamava `isValido` con il solo nome: lo chiamava su `this`, cioè sullo stesso studente
- un metodo `static` **non ha** `this`: non viene chiamato su un oggetto. In un metodo `static` di `Counter` non si può scrivere né `this` né `value`

Uso esplicito. `this` va scritto in tre casi:

1. per **distinguere** una variabile di istanza da un parametro, o da una variabile locale, che ha lo stesso nome:
   `this.max = max;`
2. per chiamare, da un costruttore, **un altro costruttore** della stessa classe:
   `this(10);`
3. per riferirsi all'**intero oggetto corrente**: per confrontarlo con un altro riferimento, passarlo a un metodo, restituirlo:
   `if (altro == this)`

Li vediamo tutti e tre in `ContoBancario`, la classe scritta insieme nella lezione 5.

L'uso esplicito di `this` – `ContoBancario.java`

```
public ContoBancario(String intestatario) {
    this.intestatario = intestatario;
    saldo = 0;
    movimenti = 0;
}

/** Un conto aperto con un primo versamento. */
public ContoBancario(String intestatario,
                     double primo) {
    this(intestatario);
    versa(primo);
}

/** false se altro e' questo stesso conto
 *  o se il prelievo fallisce. */
public boolean trasferisci(ContoBancario altro,
                           double importo) {
    if (altro == this || !preleva(importo)) {
        return false;
    }
    altro.versa(importo);
    return true;
}
```

1. `this.intestatario = intestatario;`
   Il parametro ha il nome della variabile di istanza e la *nasconde*: `intestatario` è il parametro, `this.intestatario` la variabile di istanza. Senza `this` la riga assegnerebbe il parametro a se stesso: compila, ma la variabile di istanza resta `null`.
2. `this(intestatario);`
   Chiama il primo costruttore: le inizializzazioni sono scritte una volta sola. Di norma è la prima istruzione del costruttore.
3. `altro == this`
   Il conto ricevuto è lo stesso su cui il metodo è chiamato? Nella lezione 5 `anna.trasferisci(anna, 100)` riusciva, e contava due movimenti. Ora restituisce `false`: `||` non valuta il secondo operando, e `preleva` non viene chiamato.

Nei costruttori e nei metodi `set`, dare al parametro il nome della variabile di istanza è la forma più diffusa.

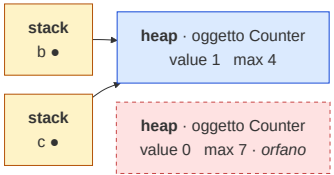
Copia di oggetti: = copia il riferimento – Copia.java

```
Counter b = new Counter(4);
Counter c = new Counter(7);

c = b;              // copia il riferimento
c.increment();
System.out.println(b.getValue());    // 1
System.out.println(c.getMax());      // 4

Counter d = new Counter(b);    // copia l'oggetto
d.increment();
System.out.println(b.getValue());    // 1
System.out.println(d.getValue());    // 2
```

- dopo `c = b` i due riferimenti portano allo **stesso oggetto**: l'incremento fatto attraverso `c` si legge anche attraverso `b`. Il contatore con `max` 7 resta orfano (lezione 5)
- lo stesso accade quando un oggetto è passato a un metodo: il parametro riceve una copia del riferimento (lezione 3)



La memoria dopo `c = b` e `c.increment()`.

Copiare un oggetto significa creare un secondo oggetto con lo stesso stato. L'assegnazione non lo fa mai, nemmeno quando `c` ha già un suo oggetto. Lo fa il **costruttore di copia** (lezione 5), una variabile di istanza alla volta: `d` è un altro oggetto.

Per la copia Java prevede anche un metodo, `clone`: lezione 8.

Confronto di oggetti: == ed equals – Counter.java (versione 5)

Aggiunti alla versione 4:

```
    public boolean equals(Counter c) {
        return max == c.max && value == c.value;
    }

    public boolean notEqual(Counter c) {
        return !equals(c);
    }
```

Nel main di Confronto:

```
Counter c = new Counter(4);
Counter d = new Counter(c);   // copia di c
Counter e = c;                // alias di c

System.out.println(c == d);           // false
System.out.println(c.equals(d));      // true
System.out.println(c == e);           // true

d.increment();
System.out.println(c.equals(d));      // false
System.out.println(c.notEqual(d));    // true
```

- `c == d` confronta i due **riferimenti**: è `true` soltanto se portano allo stesso oggetto. Vale per tutti gli oggetti, come per le `String` della lezione 2
- per confrontare lo **stato** serve un metodo, che per convenzione si chiama `equals`. È la classe a stabilire quando due suoi oggetti sono uguali: due contatori lo sono se hanno lo stesso `max` e lo stesso `value`
- `c.equals(d)` ha due operandi. Quello di sinistra è l'oggetto corrente: `max` è `this.max`. Quello di destra è il parametro
- `notEqual` non riscrive il confronto: nega `equals`. `!equals(c)` equivale a `!this.equals(c)`

Su `String`, `equals` confronta i caratteri perché la classe `String` lo definisce così. Se una classe non lo definisce, `c.equals(d)` compila lo stesso e confronta i riferimenti, come `==`: accadeva con gli array nella lezione 4. Il perché nella lezione 8.

Membri `static`: le variabili di classe

Variabile di classe (variabile `static`)

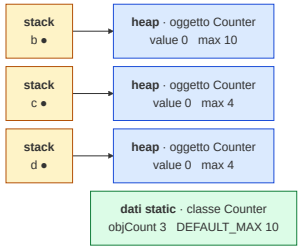
Una variabile dichiarata `static` appartiene alla classe, non ai singoli oggetti: ne esiste **una sola copia**, condivisa da tutti gli oggetti della classe.

In `Counter` (versione 5):

```
public static final int DEFAULT_MAX = 10;
private static int objCount;   // creati finora
```

- una variabile di istanza descrive un oggetto (`value`: il valore di *un* contatore); una variabile di classe descrive la classe (`objCount`: quanti contatori sono stati creati)
- la JVM la crea quando usa la classe per la prima volta, anche senza oggetti, con il valore di default o con quello scritto nella dichiarazione

`private` e `public` valgono come sempre: `objCount` è visibile solo dentro `Counter`, e i client la leggono con `getObjCount`; `DEFAULT_MAX` è a disposizione dei client come `Counter.DEFAULT_MAX`.



La memoria dopo tre `new`. Di `objCount` e `DEFAULT_MAX` c'è una sola copia, in nessun oggetto: la JVM la tiene a parte, nell'area dei dati `static` della classe.

`DEFAULT_MAX` può essere `public` perché è anche `final`: i client la leggono, nessuno la modifica. Una variabile `public static` non `final` la cambierebbe qualunque client: per questo `objCount` resta `private`.

Membri `static`: i metodi `static` – `Counter.java` (versione 5)

In `Counter`:

```
    // costruttore con argomenti
    public Counter(int m) {
        max = m;
        value = 0;
        objCount++;
    }

    // metodo static: non lavora su un oggetto
    public static int getObjCount() {
        return objCount;
    }
```

Nel `main` di `ProvaCounter`:

```
System.out.println(Counter.getObjCount());   // 0

Counter b = new Counter();
Counter c = new Counter(4);
Counter d = new Counter(c);
System.out.println(Counter.getObjCount());   // 3
System.out.println(b.getObjCount());         // 3
```

- `objCount++` sta in tutti e tre i costruttori, anche in quello di copia. Costruttori e metodi non `static` **possono** usare le variabili di classe
- un metodo `static` si chiama con il nome della classe, anche quando non esiste alcun oggetto: il primo `Counter.getObjCount()` restituisce 0
- non avendo `this`, un metodo `static` **non può** usare `value` né chiamare `increment()`: manca l'oggetto corrente. Lavora sulle variabili di classe e sui parametri, come `Math.sqrt(x)` e i metodi delle lezioni 1–4
- `b.getObjCount()` compila, ma è fuorviante: il risultato non dipende da `b`

Con cautela: una variabile di classe può essere modificata da ogni metodo della classe, su qualunque oggetto. `static` si usa quando un dato o un'operazione riguarda la classe e non un singolo oggetto.

Variabili di istanza `final` – `Counter.java` (versione 5)

In `Counter`:

```
private int value;        // valore corrente
private final int max;    // valore massimo

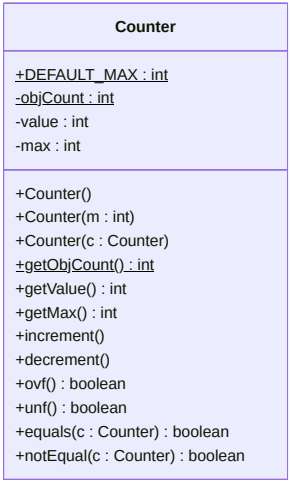
// costruttore senza argomenti
public Counter() {
    max = DEFAULT_MAX;
    value = 0;
    objCount++;
}

// setMax non c'e' piu': non compilerebbe
// public void setMax(int m) {
//     max = m;   // ERRORE: max e' final
// }
```

Nella versione 4 `setMax` lasciava al client la possibilità di portare la soglia sotto il valore già raggiunto: con `value` 3, dopo `c.setMax(1)` il contatore è in uno stato che non ha senso. La soglia di un contatore si sceglie alla creazione, e poi non deve cambiare.

- una variabile di istanza `final` riceve il suo valore **una sola volta**: nella dichiarazione, oppure in ogni costruttore, direttamente o chiamandone un altro con `this(...)`
- dopo, ogni assegnazione è un errore di compilazione, **anche dentro la classe**. Nel costruttore `max = DEFAULT_MAX` è la prima e unica assegnazione: lecita. In `setMax` sarebbe la seconda: errore
- è una costante **dell'oggetto**: ogni contatore ha il suo `max`, 10 per `b` e 4 per `c`. È il `final` delle variabili locali della lezione 2, applicato allo stato dell'oggetto
- `static final` è invece una costante **della classe**: un solo valore per tutti, scritto nella dichiarazione. Il nome è in maiuscolo: `DEFAULT_MAX`

Counter versione 5: il quadro



Variabile	Dichiarata	Quante copie	Dopo la creazione
`value`	–	una per oggetto	cambia
`max`	`final`	una per oggetto	non cambia
`objCount`	`static`	una per la classe	cambia
`DEFAULT_MAX`	`static final`	una per la classe	non cambia

Nel `main` di `ProvaCounter`:

```
System.out.println(Counter.DEFAULT_MAX);      // 10
System.out.println(b.getMax());               // 10
System.out.println(c.getMax());               // 4
// c.setMax(2);   // ERRORE: setMax non esiste
// ERRORE: objCount e' private
// Counter.objCount = 100;
// ERRORE: DEFAULT_MAX e' final
// Counter.DEFAULT_MAX = 20;
```

Nel diagramma UML i membri `static` sono sottolineati. Rispetto alla versione 4, `setMax` è scomparso: `max` è `final`.

Classi immutabili

Classe immutabile

Una classe che non offre ai client alcun modo di modificare lo stato dei suoi oggetti. I suoi oggetti si dicono *immutabili*: dopo la creazione non cambiano più.

Ne usate una dalla lezione 2: `String`. Lo sono anche le classi wrapper, come `Integer`. `s.toUpperCase()` non modifica `s`: restituisce una nuova stringa.

Che cosa si guadagna

- il riferimento a un oggetto immutabile si può **condividere** senza rischi: nessuno, attraverso un alias, può cambiare l'oggetto. Per questo le `String` si passano e si restituiscono senza mai copiarle
- le regole sullo stato si controllano una volta sola, nel costruttore: un oggetto che nasce valido resta valido

Come si scrive una classe immutabile:

- tutte le variabili di istanza sono `private final`
- nessun metodo `set`, e nessun altro metodo che modifichi lo stato
- un metodo che "trasforma" l'oggetto crea e restituisce un **nuovo** oggetto, come `toUpperCase`
- se una variabile di istanza si riferisce a un oggetto modificabile serve una cautela in più: la vediamo nelle prossime due slide

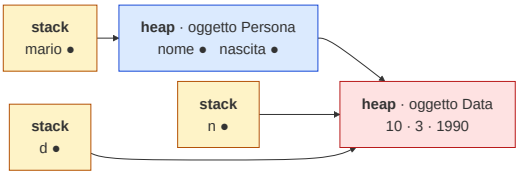
Non tutte le classi devono esserlo. `Counter` non è immutabile: contare significa cambiare stato. Ha una variabile `final`, `max`, e una che non lo è, `value`.

Il *privacy leak* – `Persona.java` (versione 1)

In `Persona`:

```
private final String nome;
private final Data nascita;

public Persona(String nome, Data nascita) {
    this.nome = nome;
    this.nascita = nascita;
}
public Data getNascita() {
    return nascita;
}
```



`Data` ha i metodi `set`: **non** è immutabile.

Nel `main` di `ProvaPersona`:

```
Data d = new Data(15, 3, 2006);
Persona mario = new Persona("Mario", d);
Data n = mario.getNascita();
n.setGiorno(10);
d.setAnno(1990);
Data ora = mario.getNascita();
System.out.println(ora.getGiorno());   // 10
System.out.println(ora.getAnno());   // 1990
```

Privacy leak

Un client ottiene un riferimento a un oggetto modificabile che fa parte dello stato `private` di un altro oggetto: può cambiare quello stato senza passare dai metodi della classe.

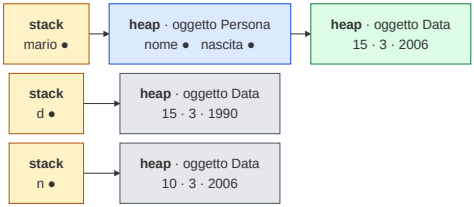
- il costruttore conserva il riferimento, `getNascita` lo restituisce: `d` e `n` portano alla `Data` di `mario`
- `nascita` è `final`, ma non basta: `final` vieta di riassegnare il **riferimento**, non di modificare l'oggetto

Il *privacy leak*: i rimedi – `Persona.java` (versione 2)

In `Persona`, al posto del costruttore e del `get` della versione 1:

```
public Persona(String nome, Data nascita) {
    this.nome = nome;
    this.nascita = new Data(nascita);    // copia
}

public Data getNascita() {
    return new Data(nascita);            // copia
}
```



Lo stesso `main` ora stampa 15 e 2006: `d` e `n` portano a oggetti diversi da quello di `mario`, e modificarli non lo tocca.

Tre modi di evitare il privacy leak

1. usare solo variabili di istanza di **tipo primitivo**: funziona, ma è una limitazione molto forte
2. non conservare e non restituire il riferimento, ma una **copia** dell'oggetto: è la versione 2, con il costruttore di copia di `Data`
3. usare solo riferimenti a oggetti **immutabili**: se `Data` non avesse i `set`, la versione 1 sarebbe già corretta. Per questo `nome`, una `String`, non dà problemi

Vale anche per gli **array**, che sono oggetti modificabili: un `get` restituisce una copia dell'array `private` (`Arrays.copyOf`, lezione 4).

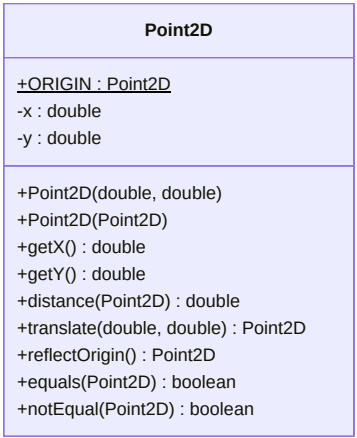
Commentiamo insieme (20 min) – il problema

Il problema. Un programma di geometria piana lavora con i punti: li sposta, ne calcola le distanze, li confronta. Serve una classe che rappresenti un punto del piano.

Le regole

- un punto ha due coordinate, x e y, e dopo la creazione **non cambia più**
- spostare un punto, o cercarne il simmetrico rispetto all'origine, significa ottenere **un altro** punto: quello di partenza resta dov'è
- di due punti si può calcolare la distanza
- due punti sono uguali se hanno le stesse coordinate
- l'origine $(0, 0)$ serve spesso: è a disposizione di tutti come costante

La soluzione è già scritta: la classe `Point2D`, che fa rispettare le regole, e il programma `ProvaPoint2D`, che la usa. Li leggiamo insieme, riga per riga, cercando `this`, `static` e `final`.



Commentiamo insieme – Point2D.java (1/2)

```java
/** Un punto del piano. Immutabile. */
public class Point2D {

    /** L'origine: una costante condivisa. */
    public static final Point2D ORIGIN = new Point2D(0, 0);

    private final double x;
    private final double y;

    public Point2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    /** Costruttore di copia. */
    public Point2D(Point2D p) {
        this(p.x, p.y);
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }
```

- `x` e `y` sono `private final`: ricevono il valore nel costruttore, e poi nessun metodo, nemmeno di `Point2D`, può cambiarle. Non c'è alcun `set`
- il costruttore di copia chiama l'altro costruttore con `this(p.x, p.y)`
- `this.x = x`: il parametro ha il nome della variabile di istanza
- `ORIGIN` è `public static final`: una costante della classe. L'oggetto è creato una volta sola, quando la JVM comincia a usare `Point2D`. I client scrivono `Point2D.ORIGIN`

Commentiamo insieme – Point2D.java (2/2)

```
    public double distance(Point2D p) {
        double dx = x - p.x;
        double dy = y - p.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    /** CREA il punto spostato di (dx, dy). */
    public Point2D translate(double dx, double dy) {
        return new Point2D(x + dx, y + dy);
    }

    /** CREA il simmetrico rispetto all'origine. */
    public Point2D reflectOrigin() {
        return ORIGIN.translate(-x, -y);
    }

    public boolean equals(Point2D p) {
        return x == p.x && y == p.y;
    }

    public boolean notEqual(Point2D p) {
        return !equals(p);
    }
}
```

Da notare, leggendo:

- in `distance`, `x` e `y` sono dell'oggetto corrente (`this.x`); `p.x` e `p.y` del punto ricevuto. `dx` e `dy` sono variabili locali
- `translate` e `reflectOrigin` non modificano nulla: creano un nuovo punto e ne restituiscono il riferimento. Non potrebbero fare altro: `x` e `y` sono `final`
- `reflectOrigin` non scrive `new`: il simmetrico di (x, y) è l'origine spostata di $(-x, -y)$, e lo chiede a `translate` chiamato su `ORIGIN`. Dentro la classe la costante si usa con il solo nome
- `equals` confronta le coordinate con `==`: fra `double` il confronto è esatto, e due punti ottenuti con calcoli diversi possono differire per un arrotondamento
- nessuno di questi metodi è `static`: ciascuno lavora sulle coordinate di un punto

Commentiamo insieme – ProvaPoint2D.java

```java
/** LEGGE il punto ricevuto e lo stampa. */
public static void stampa(String nome, Point2D p) {
    System.out.println(nome + " = (" + p.getX()
            + ", " + p.getY() + ")");
}

public static void main(String[] args) {
    Point2D p = new Point2D(3, 4);
    Point2D q = p.translate(1, 1);
    Point2D r = p.reflectOrigin().reflectOrigin();
    stampa("p", p);
    stampa("q", q);
    stampa("r", r);

    System.out.println(p.distance(Point2D.ORIGIN));
    System.out.println(p == r);
    System.out.println(p.equals(r));
    System.out.println(p.notEqual(q));

    p.translate(10, 10);
    stampa("p", p);
}
```

```
p = (3.0, 4.0)
q = (4.0, 5.0)
r = (3.0, 4.0)
5.0
false
true
true
p = (3.0, 4.0)
```

Rispondiamo insieme:

1. perché l'ultima riga stampa ancora `(3.0, 4.0)`? Che fine fa il punto creato da `p.translate(10, 10)`?
2. quanti oggetti `Point2D` crea il programma, contando `ORIGIN`? Quanti sono orfani alla fine?
3. perché `p == r` è `false` e `p.equals(r)` è `true`?
4. un client può scrivere `Point2D.ORIGIN = p`? Può spostare l'origine in un altro modo?
5. il costruttore di copia serve, in una classe immutabile?

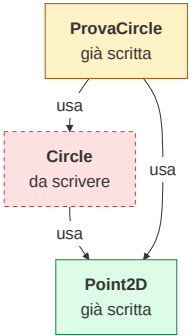
Insieme, passo passo (30 min) – il problema

Il problema. Al programma di geometria serve anche il cerchio: un centro, che è un punto del piano, e un raggio.

Le regole

- un cerchio, come un punto, dopo la creazione non cambia più
- il raggio è sempre positivo: un raggio non positivo viene sostituito da 1
- un cerchio si crea indicando il centro e il raggio, oppure soltanto il raggio: in questo caso il centro è l'origine
- di un cerchio si possono leggere il centro e il raggio, calcolare l'area, sapere se contiene un punto; il bordo fa parte del cerchio
- ingrandire o rimpicciolire un cerchio di un fattore k significa ottenere un altro cerchio, con lo stesso centro e il raggio moltiplicato per k
- due cerchi sono uguali se hanno centri uguali e lo stesso raggio
- in ogni momento si può sapere quanti cerchi sono stati creati

`Point2D` c'è già: la riusiamo senza modificarla. Anche il programma di prova, `ProvaCircle`, è già scritto: manca la classe `Circle`.



`Circle` avrà due ruoli: sarà usata da `ProvaCircle` e sarà client di `Point2D`.

Insieme, passo passo – ProvaCircle.java

```
public static void stampa(String nome, Circle c) {
    Point2D o = c.getCenter();
    System.out.println(nome + ": centro (" + o.getX()
            + ", " + o.getY() + "), raggio "
            + c.getRadius());
}

public static void main(String[] args) {
    Point2D p = new Point2D(3, 4);
    Circle a = new Circle(p, 5);
    Circle b = new Circle(2);
    Circle c = a.scale(2);
    Circle d = new Circle(new Point2D(3, 4), 5);
    Circle e = new Circle(p, -1);
    stampa("b", b);
    stampa("c", c);
    stampa("e", e);
    System.out.println(a.contains(Point2D.ORIGIN));
    System.out.println(b.contains(p));
    System.out.println(a.equals(d));
    System.out.println(a.notEqual(c));
    System.out.println(b.area());
    System.out.println(Circle.getObjCount());
}
```

Deve stampare:

```
b: centro (0.0, 0.0), raggio 2.0
c: centro (3.0, 4.0), raggio 10.0
e: centro (3.0, 4.0), raggio 1.0
true
false
true
true
12.566370614359172
5
```

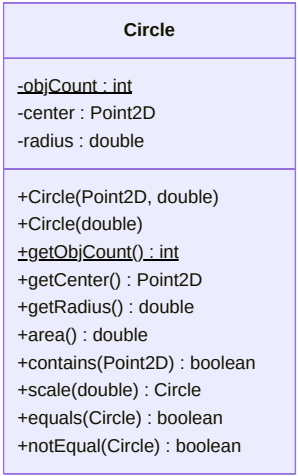
Come nella lezione 5, il client viene prima della classe: leggendolo ricaviamo l'**interfaccia** che `Circle` deve offrire.

Due indizi: `getObjCount` è chiamato sul nome della classe; `a` e `d` hanno per centro due oggetti `Point2D` diversi, con le stesse coordinate.

Insieme, passo passo – dal programma di prova alla classe

Prima di scrivere codice, decidiamo insieme:

1. quali **variabili di istanza**? → `center`, un `Point2D`, e `radius`: `private final`, perché il cerchio non cambia
2. dove si contano i cerchi? → in una variabile `static`, `objCount`: una sola per tutta la classe. La legge un metodo `static`
3. quanti **costruttori**? → due. Il controllo sul raggio e `objCount++` si scrivono in uno solo: l'altro lo chiama con `this(...)`
4. quali `get`, quali `set`? → `getCenter` e `getRadius`; nessun `set`
5. come si scrive `scale`? → non può modificare `radius`: crea un nuovo `Circle`
6. come si confrontano due cerchi? → i centri con `equals` di `Point2D`, non con `==`; i raggi con `==`
7. `getCenter` restituisce il riferimento al centro: è un privacy leak? → no, perché `Point2D` è immutabile



Una possibile soluzione (1/2) – `Circle.java`

```java
/** Un cerchio del piano. Immutabile. */
public class Circle {

    private static int objCount;   // cerchi creati

    private final Point2D center;
    private final double radius;

    /** Un raggio non positivo diventa 1. */
    public Circle(Point2D center, double radius) {
        this.center = center;
        if (radius > 0) {
            this.radius = radius;
        } else {
            this.radius = 1;
        }
        objCount++;
    }

    /** Il centro e' l'origine. */
    public Circle(double radius) {
        this(Point2D.ORIGIN, radius);
    }

    public static int getObjCount() {
        return objCount;
    }

    public Point2D getCenter() {
        return center;
    }

    public double getRadius() {
        return radius;
    }
```

- il primo costruttore fa tutto: assegna le due variabili `final`, controlla il raggio, conta il cerchio. I due rami dell'`if` assegnano `radius` una volta ciascuno: è comunque una sola assegnazione
- il secondo lo chiama con `this(...)`: il controllo e `objCount++` sono scritti in un punto solo
- `getCenter` restituisce il riferimento, senza copie: `Point2D` è immutabile

Una possibile soluzione (2/2) – `Circle.java`

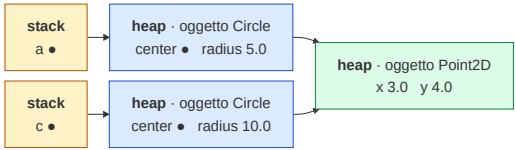
```
    public double area() {
        return Math.PI * radius * radius;
    }

    /** true se p e' nel cerchio o sul bordo. */
    public boolean contains(Point2D p) {
        return center.distance(p) <= radius;
    }

    /** CREA il cerchio con il raggio per k. */
    public Circle scale(double k) {
        return new Circle(center, radius * k);
    }

    public boolean equals(Circle c) {
        return center.equals(c.center)
                && radius == c.radius;
    }

    public boolean notEqual(Circle c) {
        return !equals(c);
    }
}
```



a e c condividono lo stesso centro, a cui porta anche p: nessun rischio, perché nessuno può modificarlo.

- `contains` chiama `distance` sul centro: `Circle` usa `Point2D` solo attraverso i suoi metodi `public`. Qui `center.x` non compila: `x` è `private` di un'altra classe
- `scale` passa dal costruttore: anche il nuovo cerchio ha un raggio valido, ed è contato
- `equals` confronta i centri con `equals` di `Point2D`: con `==` i cerchi a e d risulterebbero diversi

Riepilogo: cosa abbiamo trattato

Concetto (unità di riferimento)	**Dove compare oggi**
Il riferimento `this`, uso implicito (U10)	`getValue`, `increment`; ogni metodo non `static`
Uso esplicito di `this` (U10)	`ContoBancario`; i costruttori di `Point2D` e `Circle`
Copia di oggetti: `=` e costruttore di copia (U10)	`Copia`; `Point2D(Point2D p)`
Confronto: `==`, `equals`, `notEqual` (U10)	`Confronto`; `Point2D`; `Circle`
Variabili di classe e metodi `static` (U10)	`objCount` e `getObjCount` in `Counter` e in `Circle`
Costanti di classe, `static final` (U10)	`Counter.DEFAULT_MAX`; `Point2D.ORIGIN`
Variabili di istanza `final` (U10)	`max` in `Counter`; `x`, `y`, `center`, `radius`
`Counter` versione 5 (U10)	`ProvaCounter`
Classi immutabili (U10)	`String`; `Point2D`; `Circle`
Privacy leak e rimedi (U10)	`Persona` versioni 1 e 2; `getCenter` in `Circle`

Prossima lezione (L7): ereditarietà e polimorfismo – `extends`, `super`, ridefinizione dei metodi, classi e metodi astratti.

Homework 6 (da svolgere entro la prossima lezione)

Tre classi **immutabili**, ciascuna con il suo programma di prova (`ProvaSegment`, `ProvaFrazione`, `ProvaMisure`): variabili di istanza `private final`, nessun `set`, metodi `equals` e `notEqual`.

1. `Segment.java`: un segmento del piano, dato dai suoi estremi: due `Point2D` (la classe di oggi, da copiare nella cartella `homework/` senza modifiche). Costruttore `Segment(Point2D a, Point2D b)`; `getA()`, `getB()`; `length()`; `midpoint()` crea il punto medio; `translate(double dx, double dy)` crea il segmento spostato. Per `equals` il segmento AB e il segmento BA sono uguali. Il programma di prova riceve le coordinate degli estremi dalla riga di comando (`java ProvaSegment 0 0 3 4`) e stampa lunghezza, punto medio e confronto fra AB e BA, con `equals` e con `==`.

2. `Frazione.java`: numeratore e denominatore `int`, sempre ridotti ai minimi termini e con il denominatore positivo ($6/-8$ diventa $-3/4$). Costruttori `Frazione(int num, int den)`, con precondizione `den` diverso da 0, e `Frazione(int n)`, la frazione $n/1$; `getNum()`, `getDen()`; `somma(Frazione f)` e `prodotto(Frazione f)` creano una nuova frazione; `valore()` restituisce il `double` corrispondente; `descrizione()` una stringa come `"-3/4"`. Il massimo comune divisore lo calcola un metodo `private static`. Due costanti di classe: `ZERO` e `UNO`. Il programma di prova legge da tastiera due frazioni (quattro interi): se un denominatore è 0 stampa un messaggio e termina, altrimenti stampa le due frazioni, la somma e il prodotto.

3. `Misure.java`: una serie di misure, per esempio le temperature di una giornata, in un `double[] private final`. Costruttore `Misure(double[] valori)`, con precondizione di almeno un valore; `getQuante()`, `getValore(int i)`, `getValori()`; `media()`, `massimo()`; `aggiungi(double v)` crea una serie con una misura in più, in coda. Nessun privacy leak: né l'array passato al costruttore né quello restituito da `getValori` devono permettere al client di modificare la serie. Il programma di prova modifica entrambi gli array e stampa la serie: deve risultare immutata.

Usate solo ciò che abbiamo visto finora.

Domande?

mattia.fonisto@unina.it